Image Preprocessing and Binary Conversion

The following Python code demonstrates how to:

1. Download and extract image data.
2. Convert the image into a binary format using pixel intensity thresholding.
3. Modify the image visually using the Pillow library.

Code

```
# Install gdown and download ZIP file
!pip install --upgrade --no-cache-dir gdown
!gdown 1QTi7dJtNAfFR5mG0rd8K3ZGvEIfSn_DS
!unzip PersianData.zip

import numpy as np
from PIL import Image, ImageDraw
import random
import os
import matplotlib.pyplot as plt

def convertImageToBinary(path):
    """
    Convert an image to a binary representation based on pixel
        ↪ intensity.

    Args:
        path (str): The file path to the input image.

    Returns:
        list: A binary representation of the image where white is
            ↪ represented by -1
              and black is represented by 1.
    """
    image = Image.open(path)
    draw = ImageDraw.Draw(image)
    width, height = image.size
    pix = image.load()
    factor = 100
    binary_representation = []

    for i in range(width):
        for j in range(height):
            red, green, blue = pix[i, j][:3]
            total_intensity = red + green + blue

            if total_intensity > (((255 + factor) // 2) * 3):
                red, green, blue = 255, 255, 255
                binary_representation.append(-1)
            else:
                red, green, blue = 0, 0, 0
                binary_representation.append(1)

            draw.point((i, j), (red, green, blue))
```

```
43
44      del draw
45      return binary_representation
```

Explanation

- **Lines 1-3:** Install ‘gdown‘, download the ZIP archive from Google Drive, and extract it.
- **Lines 5-10:** Import required libraries for image handling, math, and plotting.
- **Function `convertImageToBinary`:**
  - Opens an image using PIL.
  - Reads each pixel's RGB values.
  - Calculates intensity as sum of RGB.
  - Uses a threshold value to binarize the image.
  - Changes image pixels to white (RGB: 255,255,255, encoded as -1) or black (RGB: 0,0,0, encoded as 1).
  - Modifies the image directly and returns a flattened list of binary values.

article listings xcolor tcolorbox amsmath

Generating Noisy Images in Python

The following Python code creates noisy versions of a set of image files by adding random noise to each pixel's RGB values. The processed images are then saved under new filenames.

Python Code

```
1  from PIL import Image, ImageDraw
2  import random
3
4  def generateNoisyImages():
5      # List of image file paths
6      image_paths = [
7          "/content/1.jpg",
8          "/content/2.jpg",
9          "/content/3.jpg",
10         "/content/4.jpg",
11         "/content/5.jpg"
12     ]
```

```
13
14      for i, image_path in enumerate(image_paths, start=1):
15          noisy_image_path = f"/content/noisy{i}.jpg"
16          getNoisyBinaryImage(image_path, noisy_image_path)
17          print(f"Noisy image for {image_path} generated and
                ↪ saved as {noisy_image_path}")
18
19  def getNoisyBinaryImage(input_path, output_path):
20      """
21      Add noise to an image and save it as a new file.
22
23      Args:
24          input_path (str): The file path to the input image.
25          output_path (str): The file path to save the noisy
                ↪ image.
26      """
27      # Open the input image.
28      image = Image.open(input_path)
29
30      # Create a drawing tool for manipulating the image.
31      draw = ImageDraw.Draw(image)
32
33      # Determine the image's width and height in pixels.
34      width = image.size[0]
35      height = image.size[1]
36
37      # Load pixel values for the image.
38      pix = image.load()
39
40      # Define a factor for introducing noise.
41      noise_factor = 10000000
42
43      # Loop through all pixels in the image.
44      for i in range(width):
45          for j in range(height):
46              # Generate a random noise value within the
                    ↪ specified factor.
47              rand = random.randint(-noise_factor,
                    ↪ noise_factor)
48
49              # Add the noise to the Red, Green, and Blue (RGB
                    ↪ ) values of the pixel.
50              red = pix[i, j][0] + rand
51              green = pix[i, j][1] + rand
52              blue = pix[i, j][2] + rand
53
54              # Ensure that RGB values stay within the valid
                    ↪ range (0-255).
55              if red < 0:
56                  red = 0
```

```
            if green < 0:
                green = 0
            if blue < 0:
                blue = 0
            if red > 255:
                red = 255
            if green > 255:
                green = 255
            if blue > 255:
                blue = 255

            # Set the pixel color accordingly.
            draw.point((i, j), (red, green, blue))

    # Save the noisy image as a file.
    image.save(output_path, "JPEG")

    # Clean up the drawing tool.
    del draw
```

Listing 1: Code to generate noisy images from original image files.

Explanation

[colback=white, colframe=black, title=Function: `generateNoisyImages`] This function initializes a list of 5 image paths (e.g., 1.jpg to 5.jpg). It loops through them using `enumerate`, and for each image:

- Calls `getNoisyBinaryImage()` to generate a noisy version
- Saves the new image with the prefix `noisy#.jpg`

[colback=white, colframe=black, title=Function: `getNoisyBinaryImage`] This function applies pixel-wise noise to an image:

1. Loads the image using `PIL.Image`.
2. Accesses pixel data and creates a `Draw` object to edit it.
3. For every pixel:
    - Adds a random value (from `-noise_factor` to `+noise_factor`) to each of the R, G, and B channels.
    - Clips values to the valid range [0, 255].
    - Sets the modified color to the pixel.
4. Saves the modified image to the output path.

Note: The current `noise_factor = 10000000` is very high and will turn most pixels fully black or white. Use values like 30–50 for realistic noise.

Integration Note

This code is self-contained and can be safely appended to an existing image processing project. Ensure that ‘PIL‘ and ‘random‘ are already imported in your main script.

1. Function: `convertImageToBinary`

```python
def convertImageToBinary(path: str, factor: int=100, output_path : str = None):
    """
    Convert an image to a binary representation based on pixel intensity.
    Args:
        path (str): The file path to the input image.
        factor (int, optional): Adjusts the binarization threshold (Default: 100).
    Returns:
        list: A binary representation of the image where white is -1 and black is 1.
    """
    with Image.open(path) as img:
        image = img.convert("RGB")

    img_array = np.array(image)
    intensity = img_array.sum(axis=2)
    threshold = ((255 + factor) // 2) * 3
    binary_representation = np.where(intensity > threshold, -1, 1).flatten()

    if output_path:
        binarized_array = np.where(intensity > threshold, 255, 0)
        binarized_array = np.stack([binarized_array]*3, axis=-1)
        binarized_image = Image.fromarray(binarized_array.astype("uint8"), "RGB")
        binarized_image.save(output_path, 'JPEG')

    return binary_representation
```

Listing 2: Convert image to binary array based on intensity

[colback=white, colframe=black, title=Explanation: `convertImageToBinary`] This function reads an RGB image and converts it into a binary 1D array, based on the total intensity of each pixel.

- Converts the image to a NumPy array and calculates total brightness by summing R, G, and B values.

- Compares the summed intensity to a threshold determined by `factor`.
- Returns `-1` for bright pixels and `1` for dark ones.
- If an `output_path` is given, it also saves the visual binarized image with only black and white values.

2. Function: `getNoisyBinaryImage`

```python
def getNoisyBinaryImage(input_path, output_path,
    ↪ noise_factor=10000000, random_state=None):
    """
    Add noise to an image and save it as a new file.
    Args:
        input_path (str): Input image path.
        output_path (str): Output image save path.
        noise_factor (int): Amplitude of noise (default:
            ↪ 10,000,000).
        random_state (int/None): Seed for reproducibility.
    """
    with Image.open(input_path) as img:
        image = img.convert("RGB")

    img_array = np.array(image)

    if random_state is not None:
        np.random.seed(random_state)

    rand = np.random.randint(-noise_factor, noise_factor,
        ↪ img.size[0:2])

    red   = np.clip(img_array[:, :, 0] + rand, 0, 255)
    green = np.clip(img_array[:, :, 1] + rand, 0, 255)
    blue  = np.clip(img_array[:, :, 2] + rand, 0, 255)

    noise_image = np.stack((red, green, blue), axis=2)
    noise_image = Image.fromarray(noise_image.astype("uint8"
        ↪ ), "RGB")
    noise_image.save(output_path, "JPEG")
```

Listing 3: Add noise to an image and save it

[colback=white, colframe=black, title=Explanation: `getNoisyBinaryImage`] This function injects pixel-wise random noise to the RGB channels of an image:

- Noise is generated in the range of `[-noise_factor, +noise_factor]`.
- The same noise value is added to all 3 channels (R, G, B).
- Uses `np.clip` to keep values within [0, 255].

- Optionally seeds the random generator for reproducible results.
- Saves the resulting image to disk.

3. Function: `generateNoisyImages`

```python
def generateNoisyImages(noise_factor=10000000, random_state = None):
    # List of image file paths
    image_paths = [
        "1.jpg", "2.jpg", "3.jpg", "4.jpg", "5.jpg"
    ]

    os.makedirs('noisy', exist_ok=True)
    for i, image_path in enumerate(image_paths, start=1):
        noisy_image_path = f"noisy/noisy{i}.jpg"
        getNoisyBinaryImage(image_path, noisy_image_path, noise_factor, random_state)
        print(f"Noisy image for {image_path} generated and saved as {noisy_image_path}")
```

Listing 4: Batch generation of noisy images

[colback=white, colframe=black, title=Explanation: `generateNoisyImages`] This utility function processes a batch of 5 predefined images:

- Calls `getNoisyBinaryImage` for each image.
- Stores outputs in a folder named `noisy`.
- Useful for automated dataset augmentation.

4. Function: `BinarytoImage`

```python
def BinarytoImage(binary_img):
    binary_img = np.array(binary_img)
    img = np.where(binary_img == -1, 255, 0)
    img = img.reshape(96,96)
    img = np.stack([img]*3, axis=2)
    img = Image.fromarray(img.astype("uint8"), "RGB")
    return img
```

Listing 5: Convert a binary array back to an image

[colback=white, colframe=black, title=Explanation: `BinarytoImage`] This function reconstructs a 96x96 RGB image from a binary array:

- Pixels with value `-1` are set to white (255).

- Pixels with value `1` are set to black (0).
- Expands single-channel grayscale into 3-channel RGB.
- Returns the image (but does not save it).

5. Class: `HammingNetwork`

```
class HammingNetwork:
    def __init__(self, patterns):
        self.patterns = np.array([p for p in patterns])
        self.n_classes = len(patterns)

    def predict(self, x_noisy):
        similarities = np.dot(self.patterns, x_noisy)
        return self.patterns[np.argmax(similarities)]

# Load training patterns (binary vectors) from 5 clean
    ↪ images
images = [convertImageToBinary(f'{i}.jpg') for i in range(1,
    ↪  6)]
model = HammingNetwork(images)
```

Listing 6: Define and use a Hamming network for pattern classification

[colback=white, colframe=black, title=Explanation: `HammingNetwork`] This class implements a Hamming network for pattern recognition based on similarity (dot product) between binary vectors:

- `__init__` stores the input patterns and calculates the number of classes.
- `predict()` computes the dot product (similarity) between the noisy input and all stored patterns, then returns the most similar one.
- In the example, the model is trained using binarized versions of five clean images.

6. Generate Noisy Images and Convert to Binary

```
generateNoisyImages(1100, 93)
paths = [f'noisy/noisy{i}.jpg' for i in range(1, 6)]
noise = [convertImageToBinary(f'{i}') for i in paths]
```

Listing 7: Generate noisy images and convert them to binary vectors

[colback=white, colframe=black, title=Explanation: Noisy Image Processing]

- Noisy versions of the 5 training images are generated using `generateNoisyImages()`.

- Each noisy image is then converted into a binary vector using `convertImageToBinary()`.
- These binary vectors simulate noisy inputs to be classified by the Hamming network.

7. Visualization and Prediction

```
def show_images(noise, predictions):
    fig, axes = plt.subplots(2, 5, figsize=(15, 5))
    for i in range(5):
        n = noise[i]
        p = predictions[i]
        axes[0, i].imshow(n, cmap='gray')
        axes[0, i].set_title(f"Noisy {i+1}")
        axes[0, i].axis('off')

        axes[1, i].imshow(p, cmap='gray')
        axes[1, i].set_title(f"Prediction {i+1}")
        axes[1, i].axis('off')

    plt.tight_layout()
    plt.show()

# Predict the class for each noisy binary input
per = [model.predict(noise[i]) for i in range(5)]

# Convert binary arrays back to images
noise_img = [BinarytoImage(noise[i]) for i in range(5)]
per_img = [BinarytoImage(per[i]) for i in range(5)]

# Display the noisy and predicted images side by side
show_images(noise_img, per_img)
```

Listing 8: Visualize noisy images and their Hamming predictions

[colback=white, colframe=black, title=Explanation: Prediction and Display]

- The `show_images()` function displays noisy and predicted images in a 2-row grid.
- The predictions are generated by the trained `HammingNetwork`.
- Both input noise and predicted output are converted from binary vectors back to RGB images using `BinarytoImage()`.
- This visual comparison is helpful to evaluate the robustness of the Hamming network under noise.

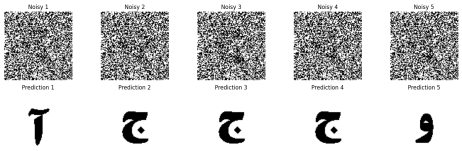

Figure 1: Caption describing the image.

8. Functions: `missimage` and `generatemissimage`

```python
def missimage(input_path, output_path, percent=20, factor=100, random_state=None):
    with Image.open(input_path) as img:
        image = img.convert("RGB")

    img_array = np.array(image)
    intensity = img_array.sum(axis=2)
    threshold = ((255 + factor) // 2) * 3
    blackpix = np.where(intensity < threshold)
    number = round(len(blackpix[0]) * percent / 100)

    if random_state is not None:
        np.random.seed(random_state)

    pixels_to_flip = np.random.choice(range(len(blackpix[0])), size=number, replace=False)
    img_array[blackpix[0][pixels_to_flip], blackpix[1][pixels_to_flip]] = 255
    img = Image.fromarray(img_array.astype("uint8"), "RGB")
    img.save(output_path, "JPEG")

def generatemissimage(percent=20, factor=100, random_state=None):
    image_paths = ["1.jpg", "2.jpg", "3.jpg", "4.jpg", "5.jpg"]
    os.makedirs('miss', exist_ok=True)
    for i, image_path in enumerate(image_paths, start=1):
        miss_image_path = f"miss/miss{i}.jpg"
        missimage(image_path, miss_image_path, percent, factor, random_state)
        print(f"Miss image for {image_path} generated and saved as {miss_image_path}")
```

Listing 9: Create synthetic missing-data images by whitening black pixels

[colback=white, colframe=black, title=Explanation: `missimage`] This function simulates partial data loss in images by randomly selecting a percentage of black pixels and converting them to white:

- Pixel intensity is calculated to identify black pixels.
- A random subset of these pixels is turned white.
- The result is saved as a new image file.

The `generatemissimage` function applies this corruption to five original images and stores them in a `miss/` directory.

9. Prediction and Visualization for Missed Images

```
generatemissimage(70, 100, 93)
paths = [f'miss/miss{i}.jpg' for i in range(1, 6)]
miss = [convertImageToBinary(i) for i in paths]

per = [model.predict(miss[i]) for i in range(5)]
miss_img = [BinarytoImage(miss[i]) for i in range(5)]
per_img = [BinarytoImage(per[i]) for i in range(5)]
show_images(miss_img, per_img)
```

Listing 10: Convert missed images to binary, predict using Hamming, and visualize

[colback=white, colframe=black, title=Explanation: Evaluation on Missed Data]

- Corrupted "miss" images are loaded and binarized.
- The Hamming network predicts which original pattern each corrupted image most closely matches.
- The `show_images` function displays the corrupted (input) and predicted (output) images for comparison.

This test shows the model's robustness to partial loss of pixel data (e.g., occlusion or dropout).

10. Visual Output of Prediction on Missed Images

11. Updated Class: `HammingNetwork` with Masking for Missing Pixels

Figure 2: Top row: Partially erased (70%) images. Bottom row: Predictions made by the Hamming network.

```python
class HammingNetwork:
    def __init__(self, patterns):
        self.patterns = np.array([p for p in patterns])
        self.n_classes = len(patterns)

    def masked_dot(self, a, b, mask):
        return np.dot(a[mask], b[mask])

    def predict(self, x_noisy):
        mask = x_noisy != -1
        similarities = [self.masked_dot(p, x_noisy, mask)
            ↪ for p in self.patterns]
        return self.patterns[np.argmax(similarities)]

# Initialize updated model
model = HammingNetwork(images)
```

Listing 11: Updated Hamming network that ignores missing (-1) pixels

[colback=white, colframe=black, title=Explanation: `HammingNetwork` with Missing Pixel Handling] This version improves upon the previous Hamming network by accounting for missing data in the input:

- The input vector `x_noisy` may contain `-1` values indicating missing pixels.
- A boolean mask is created to identify which elements are valid (not -1).
- The dot product is computed only over the valid (unmasked) pixels.
- This improves robustness for partially erased or corrupted inputs.

12. Missed Image Prediction with Masked Hamming Network

```
per = [model.predict(miss[i]) for i in range(5)]
miss_img = [BinarytoImage(miss[i]) for i in range(5)]
per_img = [BinarytoImage(per[i]) for i in range(5)]
show_images(miss_img, per_img)
```

Listing 12: Predict and visualize using Hamming with partial masking

[colback=white, colframe=black, title=Explanation: Visualization and Model Performance] This block:

- Applies the updated Hamming model to the partially erased images.
- Uses the masked similarity to predict which class each corrupted input most closely resembles.
- Displays side-by-side comparisons of corrupted inputs and predictions.

This test evaluates whether the model is able to ignore missing pixels and still make accurate predictions.

13. Final Visualization with Improved Model

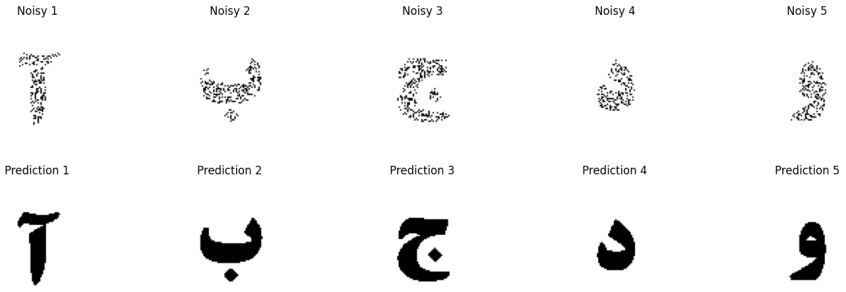

Figure 3: Top row: Missed images (70% erasure). Bottom row: Predictions using masked Hamming network.